



República Bolivariana de Venezuela

Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado P.F.G: Ingeniería en Informática

Periodo Académico: 2026-I

Unidad Curricular: Lenguajes y Compiladores

Análisis Léxico

Profesor:

Félix Marquez

Estudiantes:

Santiago Sanchez C.I: 30.001.012

Nicole García C.I: 30.809.865

Mauricio Leal C.I: 30.366.382

Jesús Ramírez C.I: 31.074.731

Ciudad Guayana, Julio 2026

Resumen

Los autómatas de pila son esenciales para reconocer lenguajes libres de contexto, como secuencias anidadas (ej. (an)), gracias a su uso de una pila como memoria temporal. Esta capacidad los hace aptos para procesar estructuras jerárquicas, como paréntesis balanceados o palíndromos, ampliamente usados en análisis sintáctico. En el ámbito del procesamiento léxico, se construyen lexers con expresiones regulares capaces de identificar tokens como números, operadores y paréntesis dentro de expresiones aritméticas. Estos lexers pueden implementarse con metacompiladores como Flex, que generan analizadores léxicos automáticos.

Un ejemplo aplicado es el diseño de un lenguaje para recetas de cocina, donde se tokenizan ingredientes, pasos y tiempos. También destacan aplicaciones en seguridad informática, como el análisis de reglas en sistemas como iptables, donde un lexer detecta comandos sospechosos o maliciosos. En conjunto, los autómatas de pila y los lexers actúan como un puente entre la teoría formal de lenguajes y su aplicación práctica en áreas como compiladores, validación de texto, lenguajes específicos de dominio y seguridad de red, demostrando su versatilidad y valor en la computación moderna.

Introducción

Los autómatas de pila son modelos fundamentales en la teoría de la computación, diseñados para reconocer lenguajes más complejos que aquellos reconocidos por los autómatas finitos, particularmente los lenguajes libres de contexto. Estos sistemas incorporan una estructura de pila que funciona como memoria dinámica, lo cual permite almacenar y recuperar información mientras procesan cadenas. Gracias a esta capacidad, gestionar construcciones anidadas como paréntesis balanceados, secuencias simétricas o etiquetas HTML bien formadas es posible, todo esto toma importancia en tareas como el análisis sintáctico en compiladores y la validación estructural de lenguajes naturales.

En paralelo, se analiza la construcción de 3 basados en expresiones regulares (regex), herramientas clave dentro del análisis léxico de lenguajes formales. Un lexer se encarga de descomponer cadenas de entrada en tokens unidades significativas como operadores, números o palabras clave facilitando el procesamiento posterior por parte del parser. Por ejemplo, en expresiones aritméticas, el lexer identifica operandos, operadores y paréntesis, y permite que el sistema interprete la expresión correctamente.

Asimismo, se explora la implementación práctica de lexers mediante metacompiladores como Flex, que generan analizadores léxicos automáticos a partir de reglas definidas con regex. Este enfoque resulta especialmente útil en el diseño de lenguajes específicos de dominio, como uno orientado a representar recetas de cocina, donde se tokenizan ingredientes, pasos y tiempos de preparación para facilitar su análisis estructurado. Además, se abordan aplicaciones en el campo de la seguridad informática, como el análisis de reglas en configuraciones de red (iptables), donde un lexer puede detectar patrones sospechosos o inconsistencias. Esta aplicación concreta evidencia cómo los fundamentos teóricos de los autómatas y del procesamiento de lenguajes se transforman en soluciones prácticas que impactan directamente en la eficiencia y seguridad de los sistemas.

Autómata de pila

Un Autómata de Pila (AP) es un modelo matemático de computación diseñado para reconocer lenguajes independientes del contexto. A diferencia de los autómatas finitos, este dispone de una memoria auxiliar en forma de pila (estructura LIFO: Last In, First Out), lo que le permite almacenar y recuperar información de manera ilimitada.

Definición práctica

El autómata de pila se define formalmente mediante los siguientes componentes (7-tupla):

- Q : Conjunto finito de estados.
 - Σ : Alfabeto de entrada (símbolos que el autómata lee).
 - Γ : Alfabeto de la pila (símbolos que pueden guardarse en la memoria).
 - δ : Función de transición (Estado, Entrada, Tope de Pila $\rightarrow$ Nuevo Estado, Operación en Pila).
 - q_0 : Estado inicial.
 - Z_0 : Símbolo inicial de la pila.
 - F : Conjunto de estados de aceptación.
-

2. Ejemplos

Ejemplo 1: El lenguaje $L = \{a^n b^n \mid n \geq 1\}$

Este lenguaje requiere que haya el mismo número de letras 'a' que de letras 'b' (ejemplo: aaabbb).

- Lógica: Por cada 'a' leída, el autómata guarda un símbolo en la pila (Push). Al empezar a leer las 'b', el autómata saca un símbolo de la pila por cada 'b' encontrada (Pop). Si al final la pila queda vacía exactamente cuando se acaban las letras, la cadena es aceptada.

Ejemplo 2: Verificación de Paréntesis

- Lógica: Es el sistema que usan los programadores. Si lee un "(", lo mete en la pila. Si lee un ")", saca el de arriba. Si al final la pila está vacía, los paréntesis están balanceados. Si sobra uno o falta uno, el autómata rechaza la cadena.
-

Importancia y comparación

El Autómata de Pila representa un salto significativo en el poder computacional respecto a los Autómatas Finitos Deterministas (AFD) y No Deterministas (AFND).

Característica	AFD / AFND	Autómata de Pila (AP)
Tipo de Lenguaje	Lenguajes Regulares.	Lenguajes Independientes del Contexto.
Memoria	Limitada (solo estados).	Infinita (en forma de Pila).
Poder de Cómputo	Bajo (no puede contar).	Alto (puede manejar recursividad).
Estructura	Lineal y simple.	Jerárquica y anidada.

2) Construcción de un Lexer para Archivos Docker

Este analizador utiliza **expresiones regulares** para identificar las directivas clave de Docker (como FROM, RUN, CMD) y distinguir entre comandos, rutas de archivos, etiquetas y comentarios.

Paso a Paso de la Construcción

1. **Definición de Patrones (Regex):** Se establecieron expresiones regulares para cada componente. Por ejemplo, las instrucciones de Docker se definen para ser reconocidas al inicio de la línea o después de un espacio.
2. **Identificación de Tokens:** Se crearon categorías de tokens como TK_INSTRUCCION, TK_COMENTARIO, TK_STRING y TK_VALOR.
3. **Lógica de Lectura:** El programa lee el archivo fuente carácter por carácter, comparando los segmentos de texto con los patrones definidos hasta el fin del archivo.
4. **Manejo de Errores:** Si un carácter no coincide con ninguna regla (como un símbolo extraño), el sistema lo reporta como un error léxico. **Código Fuente**

```
docker.cpp |
#include <iostream>
#include <string>
#include <vector>
#include <regex>
#include <iomanip>

using namespace std;

// Estructura para almacenar los tokens encontrados
struct Token {
    string tipo;
    string lexema;
    int linea;
};

void analizarDocker(string codigo) {
    // Definición de expresiones regulares para Docker
    vector<pair<string, string>> reglas = {
        {"TK_COMENTARIO",    "^#.+"},
        {"TK_INSTRUCCION",   "^(FROM|RUN|CMD|LABEL|EXPOSE|ENV|ADD|COPY|ENTRYPOINT|VOLUME|USER|WORKDIR|ARG|STOPSIGNAL|HEALTHCHECK|ONBUILD) "},
        {"TK_VALOR_ENV",      "\\b[A-Z_]+=[^\\s]+"},
        {"TK_RUTA",           "[a-zA-Z0-9._-]+|\\.[a-zA-Z0-9._-]+"},
        {"TK_STRING",         "\"[^\"]*\""},
        {"TK_IDENTIFICADOR", "[a-zA-Z0-9._-]+"},
        {"TK_ESPACIO",        "[\\s]+"}
    };

    string line;
    int num_linea = 1;
    size_t pos = 0;

    cout << left << setw(10) << "TOKEN" << setw(10) << "LEXEMA" << "LINEA" << endl;
```

docker.cpp

```

cout << left << setw(20) << "TOKEN" << setw(35) << "LEXEMA" << "LINEA" << endl;
cout << "-----" << endl;

// Procesamiento línea por línea para simular el comportamiento de un archivo Docker
char* contexto = (char*)codigo.c_str();
char* token_linea = strtok(contexto, "\n");

while (token_linea != NULL) {
    string str_linea(token_linea);
    size_t cursor = 0;

    while (cursor < str_linea.length()) {
        bool coincidencia = false;
        string sub = str_linea.substr(cursor);

        for (auto const& regla : reglas) {
            smatch match;
            if (regex_search(sub, match, regex(regla.second, regex_constants::icase))) {
                if (match.position() == 0) {
                    if (regla.first != "TK_ESPACIO") {
                        cout << left << setw(20) << regla.first
                            << setw(35) << match.str()
                            << num_linea << endl;

                        cursor += match.length();
                        coincidencia = true;
                        break;
                    }
                }
            }
        }

        if (!coincidencia) {

```

docker.cpp

```

        }

        if (!coincidencia) {
            if (!isspace(str_linea[cursor])) {
                cout << "ERROR LEXICO: Character '" << str_linea[cursor] << "' no reconocido en línea " << num_linea << endl;
                cursor++;
            }
        }

        token_linea = strtok(NULL, "\n");
        num_linea++;
    }
}

int main() {
    // Ejemplo de contenido de un archivo Docker
    string dockerfile =
        "# Imagen base\n"
        "FROM ubuntu:22.04\n"
        "RUN apt-get update\n"
        "ENV APP_HOME=/app\n"
        "WORKDIR $APP_HOME\n"
        "COPY . .\n"
        "CMD [\"python3\", \"app.py\"]\n"
        "INVALID_SYM @";

    analizarDocker(dockerfile);

    system("pause"); // Para visualizar en Dev-C++
    return 0;
}

```

Ejemplos de Ejecución (Salida Esperada)

A continuación se detallan 3 casos de prueba basados en la lógica del lexer:

Ejemplo 1: Definición de Imagen y Variables

- **Entrada:** FROM python:3.9 / ENV DEBUG=true
- **Salida:**
 - (TK_INSTRUCCION, "FROM", 1)
 - (TK_IDENTIFICADOR, "python:3.9", 1)

- (TK_INSTRUCCION, "ENV", 2)
- (TK_VALOR_ENV, "DEBUG=true", 2) **Ejemplo**

2: Comentarios y Rutas

- **Entrada:** # Instalacion / WORKDIR /usr/src/app ●

Salida:

- (TK_COMENTARIO, "# Instalacion", 1)
- (TK_INSTRUCCION, "WORKDIR", 2)
- (TK_RUTA, "/usr/src/app", 2)

Ejemplo 3: Detección de Errores Léxicos

- **Entrada:** RUN apt-get @install ●

Salida:

- (TK_INSTRUCCION, "RUN", 1)
- (TK_IDENTIFICADOR, "apt-get", 1)
- ERROR LÉXICO: Carácter '@' no reconocido en línea 1
- (TK_IDENTIFICADOR, "install", 1)

3) Definir un lenguaje L subconjunto de lenguaje Rust, construya un lexer para L utilizando metacompilador.

Manual de Usuario para el Uso del Metacompilador Flex

En el marco de la construcción de compiladores, una de las herramientas fundamentales es el generador de analizadores léxicos. Para este proyecto, he utilizado **Flex (Fast Lexical Analyzer Generator)**, una potente utilidad del proyecto GNU que nos permite generar analizadores léxicos (también conocidos como *lexers* o *scanners*) a partir de una especificación formal basada en expresiones regulares.

El propósito de este manual es documentar el proceso de instalación, configuración y uso de Flex en un entorno de desarrollo Windows, enfocado en la creación del analizador léxico para nuestro lenguaje.

1. Requisitos Previos

Para poder utilizar Flex y compilar el código que genera, es necesario contar con el siguiente software en nuestro sistema:

1. **Sistema Operativo:** Windows 7 o una versión superior.

2. **Compilador de C/C++:** Flex genera código fuente en lenguaje C.

Por lo tanto, necesitamos un compilador como GCC. La forma más sencilla de obtenerlo en Windows es a través del paquete **MinGW (Minimalist GNU for Windows)**.

3. **Editor de Texto o IDE:** Un entorno para escribir tanto el archivo de especificación de Flex (.l) como el código de prueba. Para este proyecto, he utilizado **Visual Studio Code**.

2. Proceso de Instalación y Configuración

a) Instalación del Compilador (MinGW):

1. Descargamos el instalador de MinGW desde su repositorio oficial en SourceForge.
2. Durante la instalación, seleccionamos los paquetes esenciales. Como mínimo, debemos marcar para instalación mingw32-base-bin y mingw32-gcc-g++-bin. Esto nos proporcionará los compiladores de C y C++.
3. Aplicamos los cambios y esperamos a que finalice la descarga e instalación.

b) Instalación de Flex:

1. Descargamos el paquete de Flex para Windows. Una fuente confiable es el proyecto GnuWin32. [Download GnuWin](#) [GnuWin download](#) | [SourceForge.net](#)

Ejecutamos el instalador y seguimos los pasos, aceptando la configuración por defecto.

c) Configuración de las Variables de Entorno (PATH):

Este es un paso crucial para que el sistema pueda encontrar los ejecutables de gcc, g++ y flex desde cualquier ubicación en la línea de comandos.

1. Accedemos a las "Variables de Entorno" del sistema.
2. En la sección "Variables del sistema", seleccionamos la variable Path y hacemos clic en "Editar".
3. Agregamos dos nuevas entradas con las rutas a las carpetas bin de MinGW y Flex. Típicamente, estas rutas son:
 - C:\MinGW\bin
 - C:\Program Files (x86)\GnuWin32\bin
4. Para verificar que la instalación fue exitosa, abrimos una nueva terminal (CMD o PowerShell) y ejecutamos los siguientes comandos. Deberían devolver la versión de cada programa:

```
gcc --version  
flex --version
```

3. Flujo de Trabajo con Flex

```
C:\Users\Usuario>gcc --version  
gcc (MinGW.org GCC-6.3.0-1) 6.3.0  
Copyright (C) 2016 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  
  
C:\Users\Usuario>flex --version  
flex version 2.5.4  
  
C:\Users\Usuario>_
```

El proceso para crear un lexer con Flex se resume en los siguientes pasos:

1. **Crear el Archivo de Especificación (.l):** Escribimos un archivo (ej: papitax.l) que contiene las reglas de nuestro lenguaje en forma de expresiones regulares.
2. **Generar el Código C:** Usamos Flex para procesar nuestro archivo de especificación, lo que genera un archivo C ([lex.yy.c](#)).

```
flex nombre_archivo.l
```

3. Compilar el Lexer: Compilamos el archivo C generado utilizando GCC. El flag `-lfl` enlaza la librería de Flex, que es necesaria para el funcionamiento del lexer.

```
gcc lex.yy.c -lfl -o nombre_ejecutable
```

4. Ejecutar el Lexer: Ejecutamos el programa compilado, pasándole un archivo de código fuente escrito en nuestro lenguaje como entrada.

```
./nombre_ejecutable archivo_de_prueba.txt
```

1. Descripción del Lenguaje L (Subconjunto de Rust)

Para este proyecto, se ha diseñado el lenguaje **L-Rust**, un subconjunto simplificado del lenguaje de programación **Rust**. Este lenguaje permite la declaración de variables, operaciones aritméticas básicas y estructuras de control elementales, manteniendo la sintaxis estricta de Rust.

Componentes del Lenguaje L-Rust:

- **Palabras Reservadas:** `fn`, `let`, `mut`, `if`, `else`, `while`, `return`, `i32`, `bool`, `true`, `false`.
- **Identificadores:** Secuencias de letras y números que comienzan con una letra o guion bajo (ej. `variable_1`, `contador`).
- **Literales:** Números enteros (42, 100) y valores booleanos (`true`, `false`).

- **Operadores:** +, -, *, /, =, ==, !=, <, >.
- **Delimitadores:** Llaves { }, paréntesis (), punto y coma ;, dos puntos :, y flecha de retorno ->.

2. Documentación del Proceso de Creación del Lexer (Archivo .I)

El lexer se construyó utilizando el metacompilador **Flex**. Se definieron expresiones regulares para capturar cada token del lenguaje **L-Rust**. A continuación, se presenta el código fuente del archivo `lexer_rust.l` que debes incluir en tu repositorio:

```

3  %{
4  #include <stdio.h>
5  %}
6
7  /* Definiciones de expresiones regulares */
8  DIGITO    [0-9]
9  LETRA     [a-zA-Z_]
10 ID        {LETRA}{LETRA}{DIGITO}*
11 NUMERO    {DIGITO}+
12 ESPACIO   [ \t\n\r]+
13
14 %%
15
16 "fn"      { printf("TOKEN_PALABRA_RESERVADA: fn\n"); }
17 "let"     { printf("TOKEN_PALABRA_RESERVADA: let\n"); }
18 "mut"     { printf("TOKEN_PALABRA_RESERVADA: mut\n"); }
19 "if"      { printf("TOKEN_PALABRA_RESERVADA: if\n"); }
20 "else"    { printf("TOKEN_PALABRA_RESERVADA: else\n"); }
21 "while"   { printf("TOKEN_PALABRA_RESERVADA: while\n"); }
22 "return"  { printf("TOKEN_PALABRA_RESERVADA: return\n"); }
23 "i32"     { printf("TOKEN_TIPO_DATO: i32\n"); }
24 "bool"    { printf("TOKEN_TIPO_DATO: bool\n"); }
25 "true"    { printf("TOKEN_BOOLEANO: true\n"); }
26 "false"   { printf("TOKEN_BOOLEANO: false\n"); }
27
28 {ID}      { printf("TOKEN_IDENTIFICADOR: %s\n", yytext); }
29 {NUMERO}  { printf("TOKEN_LITERAL_ENTERO: %s\n", yytext); }
30
31 "+"       { printf("TOKEN_OPERADOR: +\n"); }
32 "-"       { printf("TOKEN_OPERADOR: -\n"); }
33 "*"       { printf("TOKEN_OPERADOR: *\n"); }
34 "/"       { printf("TOKEN_OPERADOR: /\n"); }
35 "=="      { printf("TOKEN_OPERADOR: ==\n"); }
36 "!="      { printf("TOKEN_OPERADOR: !=\n"); }
37 "="      { printf("TOKEN_OPERADOR: =\n"); }
38 "<"       { printf("TOKEN_OPERADOR: <\n"); }
39 ">"       { printf("TOKEN_OPERADOR: >\n"); }
40 "->"     { printf("TOKEN_DELIMITADOR: ->\n"); }
41

```

```

41
42  "{"      { printf("TOKEN_DELIMITADOR: {\n"); }
43  "}"      { printf("TOKEN_DELIMITADOR: }\n"); }
44  "("      { printf("TOKEN_DELIMITADOR: (\n"); }
45  ")"      { printf("TOKEN_DELIMITADOR: )\n"); }
46  ";"      { printf("TOKEN_DELIMITADOR: ;\n"); }
47  ":"      { printf("TOKEN_DELIMITADOR: :\n"); }
48
49  {ESPACIO} { /* Ignorar espacios en blanco */ }
50
51  .        { printf("ERROR LÉXICO: Carácter no reconocido: %s\n", yytext); }
52
53  %%
54
55  int main(int argc, char **argv) {
56      if (argc > 1) {
57          yyin = fopen(argv[1], "r");
58          if (!yyin) {
59              printf("Error al abrir el archivo.\n");
60              return 1;
61          }
62      }
63      yylex();
64      return 0;
65  }
66
67  int yywrap() {
68      return 1;
69  }

```

3. Pasos de Instalación e Implementación (Markdown para el Repositorio)

Para el archivo README.md de tu repositorio, utiliza estos pasos detallados:

Guía de Ejecución del Lexer L-Rust Requisitos

Previos:

- Tener instalado **Flex** y un compilador de C (como **GCC** a través de MinGW).
- Configurar las variables de entorno (PATH) según el manual adjunto en el informe.

Pasos para Compilar y Ejecutar:

1. **Generar el Analizador:** Abra una terminal en la carpeta del proyecto y ejecute:

```

2
3 flex lexer_rust.l
4

```

Esto generará un archivo llamado **lex.yy.c**.

2. **Compilar el Código C:** Compile el archivo generado con GCC:

```
1  
2 gcc lex.yy.c -o lexer_rust  
3
```

3. **Probar el Lexer:** Cree un archivo de prueba llamado prueba.rs con contenido de Rust, por ejemplo:

```
1  
2 let mut x: i32 = 5;  
3 if x > 0 { return true; }  
4
```

Ejecute el lexer pasándole el archivo:

```
1  
2 ./lexer_rust prueba.rs  
3
```

4. Reflexione e indague en qué lenguajes en el área de seguridad informática, pudiera aplicar un FLEX, presente el lenguaje y su respectiva tokenización.

En el área de seguridad informática, Flex puede ser una herramienta muy útil para analizar archivos de configuración, logs, comandos maliciosos o scripts, ya que permite construir analizadores que detectan patrones de texto en tiempo real, un analizador léxico creado con Flex puede:

- Detectar comandos sospechosos en archivos de log.
- Leer reglas de firewall o configuraciones de red.
- Analizar scripts para identificar inyecciones de código.
- Detectar expresiones regulares usadas por malware.

4.1) Lenguaje aplicado para seguridad informática: Reglas de iptables

Es una herramienta ampliamente usada para configurar reglas de filtrado de paquetes en sistemas, estas reglas sirven para la seguridad del sistema.

4.2) Palabras clave del lenguaje (iptables):

-A, -P, -I → tipos de operación (append, policy, insert)

INPUT, OUTPUT, FORWARD → cadenas

-s, -d → origen y destino IP

--dport, --sport → puertos []

-j → acción (ACCEPT, DROP, REJECT)

4.3) Tokenización

```
USER@DESKTOP-ER1PRSU MSYS /c/Users/USER/Desktop
$ ./inf inf.txt
TOKEN: COMANDO, Lexema: iptables
TOKEN: OPERACION, Lexema: -A
TOKEN: CADENA, Lexema: INPUT
TOKEN: IP_OPCION, Lexema: -s
TOKEN: IP, Lexema: 192.168.1.1
TOKEN: ACCION_OPCION, Lexema: -j
TOKEN: ACCION, Lexema: ACCEPT
```

4.4) Reflexión

Es importante utilizar este tipo de lenguajes en este ámbito de seguridad informática ya que aparte que ayuda a prevenir, también a organizar y tener una estructura entendible para las personas que quieren personalizar sus instrucciones, etc. Además, les permite expandir aún más allá de lo que es scripts, etc. Porque puede también detectar expresiones y demás, si se “entrena” bien a una máquina con un lenguaje L de lexer puede lograr muchísimas cosas, fuera del ámbito de seguridad informática también es ampliamente usado por plataformas como Discord para la programación de los bots que hacen que esta plataforma de red social a nivel mundial funcione y todo esto por la misma razón de que se utilizan lexer o lenguajes programables donde el creador/owner del bot puede personalizar sus interacciones con los usuarios de una forma personalizada siguiendo un conjunto de reglas.

Conclusión

Como resultado del trabajo realizado, nos permitió comprender de forma profunda y aplicada la importancia del análisis léxico en el proceso de compilación, a lo largo de esta investigación, pudimos poner en práctica los conceptos teóricos de autómatas finitos, gramáticas regulares y gramáticas libres del contexto se integran en la práctica al construir analizadores léxicos, tanto la implementación manual desde cero como el uso de metacompiladores como flex, demostraron ser herramientas muy útiles en el ámbito de nuestra carrera y complementarias a ella según el contexto de uso, iniciamos abordando el poder de los autómatas, su construcción a partir de gramáticas regulares y su aplicación directa para reconocer patrones léxicos, luego, se logró traducir estos modelos teóricos en soluciones concretas: Primero diseñamos un lexer utilizando flex y se ejecutó desde el entorno MSYS2 MINGW64, el cual simula un entorno similar a Linux pero funciona sobre Windows y gracias a esta herramienta se pudo automatizar el proceso de análisis léxico sin necesidad de usar Python, lo cual facilitó la detección de errores léxicos en una entrada definida por un lenguaje L, en esta

implementación se mostró la funcionalidad del lexer para la primera etapa del compilador.